Universidad Continental

PRODUCTO ACADEMICO 04

EVALUACION FINAL

PA_03 Lenguaje de Programación 1

SECCIÓN:

NRC 38140

Integrantes

Eduardo Andre Flores Enao.

Grupo C

Perú, 2026

Repositorio del programa:

https://github.com/AlambrezFE/PA04_lenguaje-programacion-1-programa-sistema-de-gestion.git

INTRODUCCION:

Python es un lenguaje de programación de propósito general, interpretado y de sintaxis clara, cercana al lenguaje natural, lo que lo ha vuelto una de las puertas de entrada más habituales a la programación. Su librería estándar ofrece módulos como **datetime**, **abc**, **json** y **unittest**, suficientes para construir un sistema completo sin instalar nada adicional. Esa misma versatilidad lo hace útil fuera del aula: se usa en desarrollo web, análisis de datos, automatización e inteligencia artificial.

Para este proyecto describe un sistema de gestión para una biblioteca universitaria, escrito en Python y ejecutado desde la consola. El sistema registra libros, revistas y estudiantes, controla los préstamos y las devoluciones, y permite consultar el estado de un material y el historial de cada estudiante, para lo cual se construyó en tres versiones sucesivas donde se implementan las funcionalidades necesarias para su funcionamiento y escalabilidad.

OBJETIVOS:

- Demostrar su capacidad para analizar un problema y diseñar una solución orientada a objetos.
- Implementar las clases diseñadas aplicando los fundamentos de programación en Python.

REQUERIMIENTOS FUNCIONALES:

Parte 1: crear los requerimientos funcionales, restricciones y diagrama de las clases a utilizar.

El sistema cumple cinco requerimientos funcionales, y cada uno corresponde a una operación del menú de consola.

N.º	Requerimiento
RF1	El sistema debe permitir registrar nuevos libros en el catálogo general, con título, autor, ISBN y cantidad de ejemplares.
RF2	El sistema debe permitir registrar nuevos estudiantes, con nombre completo, código de estudiante y carrera.

N.º	Requerimiento
RF3	El sistema debe permitir prestar un libro a un estudiante verificado, registrando la fecha de salida y la fecha estimada de devolución, y reduciendo el stock disponible.
RF4	El sistema debe permitir procesar la devolución de un libro, pasando el préstamo a finalizado y reponiendo el stock del ejemplar.
RF5	El sistema debe permitir consultar el estado de un libro (disponible o prestado) y el historial de préstamos de un estudiante.

RESTRICCIONES:

Tres restricciones protegen la coherencia de los datos y evitan operaciones inválidas. Todas se validan con estructuras condicionales y se comunican mediante excepciones personalizadas.

N.º	Restricción	Motivo
R1	Un libro no puede prestarse si su stock disponible es cero.	Evita préstamos de ejemplares físicos que no existen.
R2	El código de estudiante y el ISBN deben ser únicos.	Evita registros duplicados y ambigüedad en las consultas. La misma regla protege el ISSN de las revistas.
R3	Un estudiante no puede pedir un nuevo préstamo si tiene préstamos vencidos	Obliga a devolver antes de volver a prestar.

DISEÑO ORIENTADO A OBJETOS:

El diseño reparte el problema en tres responsabilidades: qué se presta (el material), quién lo recibe (Estudiante) y el acto de prestar (Prestamo), y deja a Biblioteca la tarea de coordinarlas.

El núcleo pedido (Libro, Estudiante y Prestamo) proviene de la versión 1. La clase abstracta MaterialBibliografico y la clase Revista se añadieron en la versión 2, y la versión final solo reorganizó el código en archivos. El diagrama de clases siguiente muestra el diseño final.

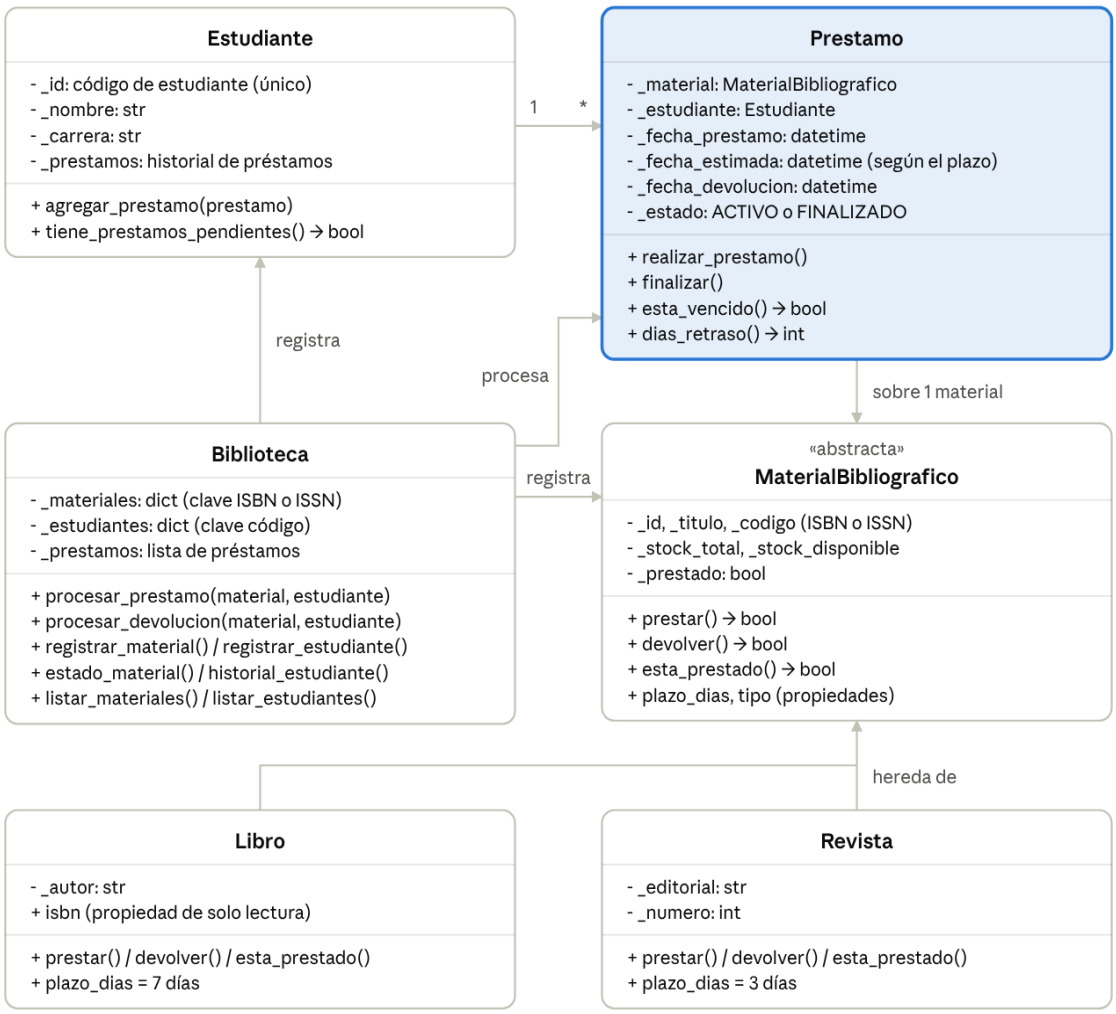


diagrama de clases del sistema de gestión de una biblioteca

Cada flecha apunta a la clase de la que depende la otra. **Libro** y **Revista** heredan de **MaterialBibliografico**, por lo que **Prestamo** y **Biblioteca** solo necesitan conocer esa clase abstracta.

IMPLEMENTACION DEL PROGRAMA:

Parte 2: implementación de las 3 clases y primera versión del programa:

La primera versión cubrió los cinco requerimientos y las tres restricciones con cuatro clases (Libro, Estudiante, Prestamo y Biblioteca) en un único archivo.

- Libro usa atributos privados con properties, controla su stock y pasa a estado "prestado" cuando se agota el último ejemplar.
- Prestamo calcula la fecha estimada con el módulo datetime y un plazo fijo de 7 días.
- Las validaciones usan estructuras condicionales (if/else) y los métodos devuelven una tupla (éxito, mensaje).
- Biblioteca guarda libros y estudiantes en diccionarios indexados por ISBN y por código, lo que asegura la unicidad.
- Límites: solo existían libros, el plazo era único y el menú, los datos y las reglas convivían en el mismo archivo.

Parte 3: implementación de interface, polimorfismo, herencia y manejo de excepciones:

Esta versión cambió la arquitectura para que el sistema aceptara nuevos tipos de material sin reescribir su lógica.

- **Clase abstracta:** MaterialBibliografico (con ABC y @abstractmethod) actúa como interfaz. Libro hereda de ella y se añade Revista con plazo de 3 días. Escalabilidad: un tipo nuevo solo requiere una clase hija.
- **Polimorfismo:** Prestamo obtiene el plazo de material.plazo_dias, y “Biblioteca.procesar_prestamo(material, estudiante) -> bool” funciona igual con libros y revistas.
- **Excepciones:** se crearon MaterialYaPrestadoError, DatosInvalidosError y EstudianteConPendientesError. Prestamo las lanza y Biblioteca las

captura con try-except, devolviendo False y dejando el motivo en ultimo_mensaje. También se controlan los valores None.

- **Cambio que afecta al uso:** las tuplas (éxito, mensaje) se reemplazaron por excepciones, y el registro pasó de registrar_libro( ) a registrar_material(objeto). El código de la versión 1 deja de ser compatible.
- **Datos más estrictos:** los setters ahora lanzan DatosInvalidosError en lugar de ignorar un valor vacío, y procesar_prestamo() verifica que material y estudiante estén registrados.
- **Calidad:** se añadieron type hints, cumplimiento de PEP8 y el archivo test_sistema.py con 27 pruebas con **unittest**.

Versión final: Agregar detalles y modificar aspectos del código.

La última versión no cambió las reglas de negocio; reorganizó el código para que sea más legible y fácil de ampliar.

- **Nueva función:** la opción 9 del menú lista los estudiantes registrados e indica quiénes tienen préstamos pendientes
- **Separación en archivos:** main.py es el único que se ejecuta. menu.py contiene la interacción, biblioteca.py, material.py, estudiante.py y prestamo.py tienen una clase (o grupo) cada uno, y excepciones.py y validaciones.py reúnen lo compartido.
- **Menú extensible:** las opciones están en una tabla (OPCIONES) donde cada una apunta a una función; agregar una opción requiere escribir una función y una línea.
- **Dataset externo:** datos.json contiene 6 materiales, 4 estudiantes y 3 préstamos ficticios, que datos.py carga al iniciar. Los préstamos se definen como "días atrás", de modo que siempre hay un préstamo vencido para probar. Si el archivo falta o es inválido, el programa arranca vacío con un aviso.
- **Ajustes técnicos:** Estudiante consulta **prestamo.esta_activo()** para evitar una importación circular con **Prestamo, y procesar_prestamo()**

acepta una fecha opcional para cargar préstamos históricos sin romper su firma.

- **Documentación y pruebas:** se añadió un README.md y las pruebas pasaron a 34, incluyendo el listado de estudiantes y el dataset.

LIMITACIONES ACTUALES:

- Los datos viven en memoria: al cerrar el programa se pierden los registros hechos durante la sesión, y solo datos.json se vuelve a cargar.
- Como la regla R3 solo bloquea los vencidos, un estudiante puede acumular varios préstamos en plazo; fijar un tope exigiría ampliar tiene_prestamos_vencidos().
- La interfaz es de consola, aunque la lógica está separada del menú y podría reutilizarse con otra interfaz más gráfica, en un futuro aañadir un framework o una extensión de Python para añadir interfaz por aplicativo web.

CONCLUSIONES

Se ha logrado identificar cinco requerimientos funcionales y tres restricciones fundamentales para el programa de gestión, ya que cada uno se tradujo en una responsabilidad concreta de una clase: Libro y Revista controlan su stock, Estudiante conoce su historial, Prestamo valida y registra el préstamo, y Biblioteca coordina todo y garantiza la unicidad de los códigos. La evolución del proyecto nos confirma que el diseño es sólido y escalable para añadir mas funcionalidades o extensiones sea de Python o de librerías externas.

La implementación usó atributos privados con properties, condicionales para las validaciones, el módulo datetime para las fechas, una clase abstracta con ABC, herencia, polimorfismo y excepciones personalizadas manejadas con try-except. El código incluye type hints, cumple PEP8 y está comentado. Sus 34 pruebas automáticas pasan, y la ejecución con los datos de ejemplo mostró que el programa rechaza correctamente un material ya prestado y a un estudiante con préstamos vencidos.